# Formation Engine Methodology

Section 11 — Deterministic Pipeline and Self-Audit

Formation Engine Development Team

Version 3.0.0 — As Implemented in Codebase

## 1 Introduction: Reproducibility as First Principle

Most computational chemistry frameworks treat reproducibility as a nice-to-have feature. This framework treats it as a **non-negotiable requirement**. Every formation produced by this engine must be bit-identical across runs, platforms, and library versions when given the same inputs.

This section documents the **self-audit infrastructure** that enforces this requirement:

1. **Deterministic execution** (seed control, IEEE 754 compliance)
2. **Failure classification** (autonomous categorization of 10,000+ runs)
3. **Coverage-driven exploration** (gap targeter filling parameter space)
4. **Regression detection** (compare parameter changes, flag invariant breaks)

Everything documented here **exists in the codebase** with Python implementations, not theoretical proposals.

## 2 Reproducibility Requirements

### 2.1 The Determinism Contract

Given:

$$(\text{formula},\ \text{seed},\ \text{parameters}) \tag{1}$$

the engine must produce **bit-identical output**. Not "approximately the same." Not "statistically equivalent." Bit-identical.

### 2.2 The Four Pillars

**1. Seed-Controlled RNG** Every stochastic operation uses `std::mt19937` with explicit seed:

```
std::mt19937 rng(seed);
std::normal_distribution<double> dist(0.0, sigma);
for (uint32_t i = 0; i < N; ++i) {
    v[i] = dist(rng);
}
```

The same seed always produces the same random sequence.

**2. Index-Ordered Force Evaluation** Force accumulation must not depend on thread execution order:

```
// WRONG: race condition on F[i]
#pragma omp parallel for
for (int i = 0; i < N; ++i) {
    for (int j = i+1; j < N; ++j) {
        Vec3 f = compute_force(i, j);
        F[i] += f;   // RACE
        F[j] -= f;   // RACE
    }
}

// RIGHT: per-thread accumulation
#pragma omp parallel
{
    std::vector<Vec3> F_local(N, {0,0,0});
    #pragma omp for
    for (int i = 0; i < N; ++i) {
        for (int j = i+1; j < N; ++j) {
            Vec3 f = compute_force(i, j);
            F_local[i] += f;
            F_local[j] -= f;
        }
    }
    #pragma omp critical
    for (int i = 0; i < N; ++i) {
        F[i] = F[i] + F_local[i];
    }
}
```

**3. Deterministic Floating-Point**

- **No fast-math flags:** `-ffast-math` allows reordering of floating-point operations, breaking reproducibility
- **IEEE 754 compliance:** `-fno-finite-math-only`, `-fno-associative-math`
- **Explicit rounding modes:** Use default (round-to-nearest-even)
- **Avoid platform-specific intrinsics:** `rsqrt()` may differ between CPUs

**4. Fixed Library Versions** The engine pins exact versions:

- Eigen 3.4.0 (not "latest")
- ImGui 1.89.2 (not "master")
- C++17 standard (not C++20, which changes `constexpr` rules)

Version drift across runs violates determinism.

### 2.3 Validation Test

**Test protocol:**

1. Run $H_2O$ formation with seed 42, parameters $\{T = 300\text{K}, \rho = 1.0\text{g/cm}^3\}$
2. Record final geometry (64-bit binary dump of positions)
3. Repeat 10 times on different machines (Windows, Linux, macOS)
4. Compute SHA-256 hash of each geometry

**Pass criterion:** All 10 hashes are identical.
**Current status:** Pass on same platform, *not yet validated* across OS (floating-point library differences).

## 3 Failure Classifier

**Implementation:** `tools/failure_classifier.py`, 250 lines
The failure classifier categorizes every failed simulation into one of six buckets using regex pattern matching on error messages.

### 3.1 The Six Categories

From `failure_classifier.py`, lines 14–19:

```
class FailureCategory(Enum):
    NUMERICAL = "numerical"      # dt too big, overflow, NaN
    PHYSICS = "physics"          # clash explosion, unphysical bonds
    OOD = "out_of_domain"        # composition/scale outside training
    CONVERGENCE = "convergence"  # optimizer failed to converge
    TIMEOUT = "timeout"          # exceeded time limit
    UNKNOWN = "unknown"          # unclassified
```

### 3.2 Pattern Matching Rules

Lines 34–58:

```
PATTERNS = {
    FailureCategory.NUMERICAL: [
        r"(?i)nan|inf|overflow|underflow",
        r"(?i)timestep.*too.*large",
        r"(?i)numerical.*unstable",
        r"(?i)division.*by.*zero",
        r"(?i)sqrt.*negative",
    ],
    FailureCategory.PHYSICS: [
        r"(?i)clash|explosion|overlap",
        r"(?i)unphysical.*bond",
        r"(?i)energy.*diverged",
        r"(?i)force.*too.*large",
        r"(?i)invalid.*geometry",
    ],
    FailureCategory.OOD: [
```

```
        r"(?i)unknown.*element",
        r"(?i)unsupported.*composition",
        r"(?i)atom.*count.*exceeded",
    ],
    FailureCategory.CONVERGENCE: [
        r"(?i)failed.*converge",
        r"(?i)max.*iterations",
        r"(?i)optimizer.*failed",
    ],
    FailureCategory.TIMEOUT: [
        r"(?i)timeout|time.*limit|killed",
    ],
}
```

### 3.3 Classification Algorithm

Lines 71–85:

```
def classify(self, error_message: str, stderr: str,
             returncode: int) -> FailureCategory:
    """Classify␣failure␣based␣on␣error␣patterns."""

    # Timeout is special (returncode)
    if returncode in [-9, 124, 137]:  # SIGKILL, timeout
        return FailureCategory.TIMEOUT

    # Combine error sources
    full_error = f"{error_message}\n{stderr}"

    # Match patterns
    for category, patterns in self.PATTERNS.items():
        for pattern in patterns:
            if re.search(pattern, full_error):
                return category

    return FailureCategory.UNKNOWN
```

### 3.4 Minimal Reproduction Command

Every failure stores a minimal reproduction command (lines 102–112):

```
def generate_minimal_repro(self, seed: int, formula: str,
                           params: Dict) -> str:
    """Generate␣minimal␣reproduction␣command."""

    cmd_parts = [
        f"./build/vsepr-sim",
        f"--formula␣{formula}",
        f"--seed␣{seed}",
    ]

    for key, val in params.items():
        cmd_parts.append(f"--{key}␣{val}")
```

```
    return "␣".join(cmd_parts)
```

**Example output:**

```
./build/vsepr-sim --formula NaCl --seed 12345 --temp 5000 --dt 10
```

This allows instant reproduction of any failure without searching through logs.

### 3.5 The UNKNOWN Budget

The UNKNOWN category must remain below 5% of total failures. If it exceeds this threshold, the pattern matchers need updating.

**Rationale:** A large UNKNOWN fraction means the classifier is missing common failure modes, reducing its diagnostic value.

### 3.6 Output Format

The classifier produces a JSON report:

```
{
  "total_runs": 10000,
  "total_failures": 1247,
  "failures_by_category": {
    "numerical": 342,
    "physics": 198,
    "out_of_domain": 67,
    "convergence": 521,
    "timeout": 89,
    "unknown": 30
  },
  "unknown_fraction": 0.024,
  "failures": [
    {
      "category": "numerical",
      "reason": "NaN␣in␣force␣evaluation␣at␣step␣127",
      "seed": 12345,
      "formula": "H2O",
      "params": {"temp": 300, "dt": 0.001},
      "minimal_repro": "./build/vsepr-sim␣--formula␣H2O␣--seed␣12345␣..."
    },
    ...
  ]
}
```

## 4 Gap Targeter (Coverage-Driven Exploration)

**Implementation:** `tools/gap_targeter.py`, 200 lines

The gap targeter partitions parameter space into a 3D grid and schedules runs to fill sparse regions.

### 4.1 The Parameter Grid

From `gap_targeter.py`, lines 38–40:

```
# Define grid
self.scale_bins = [2, 5, 10, 20, 50, 100]  # atom count
self.temp_bins = np.linspace(50, 500, 10)   # K
self.density_bins = np.linspace(0.001, 0.1, 10)  # g/cm^3
```

- **Scale:** 6 bins (2, 5, 10, 20, 50, 100 atoms)
- **Temperature:** 10 bins (50 K to 500 K)
- **Density:** 10 bins (0.001 to 0.1 g/cm$^3$)

Total: $6 \times 10 \times 10 = 600$ cells.

### 4.2 Cell Representation

Lines 10–30:

```
@dataclass
class Cell:
    """Cell in parameter space grid."""
    scale: int
    temp: float
    density: float

    # Coverage metrics
    n_runs: int = 0
    n_success: int = 0
    n_failures: int = 0

    # Mismatch metrics
    mean_energy: float = 0.0
    std_energy: float = 0.0
    max_mismatch: float = 0.0  # vs neighbors

    def is_sparse(self, threshold: int = 10) -> bool:
        return self.n_runs < threshold

    def is_high_mismatch(self, threshold: float = 0.5) -> bool:
        return self.max_mismatch > threshold
```

### 4.3 Gap Identification

A cell is considered a **gap** if:

**Condition 1: Sparse coverage**

$$n_{\text{runs}} < 10 \tag{2}$$

**Condition 2: High mismatch with neighbors**

$$\max_{j \in \text{neighbors}} \frac{|\bar{E}_i - \bar{E}_j|}{\sqrt{\sigma_i^2 + \sigma_j^2}} > 0.5 \tag{3}$$

The mismatch metric detects discontinuities in the energy landscape (phase transitions, force field breakdowns).

## 4.4 Mismatch Computation

Lines 79–100:

```
def compute_mismatches(self):
    """Compute␣mismatch␣between␣neighboring␣cells."""

    for (i, j, k), cell in self.cells.items():
        if cell.n_success < 2:
            continue

        # Check 6 neighbors (1 in each dimension)
        neighbors = [
            (i-1, j, k), (i+1, j, k),
            (i, j-1, k), (i, j+1, k),
            (i, j, k-1), (i, j, k+1),
        ]

        mismatches = []
        for neighbor_key in neighbors:
            if neighbor_key in self.cells:
                neighbor = self.cells[neighbor_key]
                if neighbor.n_success >= 2:
                    # Energy mismatch
                    delta = abs(cell.mean_energy - neighbor.mean_energy)
                    combined_std = np.sqrt(cell.std_energy**2
                                           + neighbor.std_energy**2)
                    if combined_std > 0:
                        mismatch = delta / combined_std
                        mismatches.append(mismatch)

        if mismatches:
            cell.max_mismatch = max(mismatches)
```

## 4.5 Run Scheduling

Lines 102–125:

```
def schedule_runs(self, n_runs: int) -> List[Dict]:
    """Generate␣run␣schedule␣to␣fill␣gaps."""

    gaps = self.identify_gaps()

    if not gaps:
        print("No␣gaps␣detected␣-␣coverage␣is␣complete!")
```

```
        return []

    print(f"Identified {len(gaps)} gap cells")
    print(f"Scheduling {n_runs} runs to improve coverage...")

    schedule = []
    for i in range(n_runs):
        # Round-robin through gaps
        cell = gaps[i % len(gaps)]

        run_config = {
            "scale": cell.scale,
            "temp": cell.temp,
            "density": cell.density,
            "formula": self._generate_formula(cell.scale),
            "seed": i + 1000000,  # Unique seed
        }

        schedule.append(run_config)

    return schedule
```

### 4.6 Convergence Criterion

Coverage is considered **complete** when:

$$\frac{n_{\text{cells with } n_{\text{runs}} \geq 10}}{600} > 0.95 \tag{4}$$

95% of cells have at least 10 successful runs.

### 4.7 Usage Example

```
# Initialize gap targeter
python tools/gap_targeter.py --output results/coverage.json

# Run 1000 iterations
for i in {1..1000}; do
    # Get next batch of runs
    python tools/gap_targeter.py --schedule 10 > batch.json

    # Execute batch
    while read run; do
        eval $run
    done < batch.json

    # Update coverage
    python tools/gap_targeter.py --update results/run_$i.json
done

# Generate coverage report
python tools/gap_targeter.py --report > coverage_report.txt
```

# 5 Regression Detector

**Implementation:** `tools/regression_detector.py`, 200 lines

The regression detector reruns a benchmark suite when parameters change and reports:

- Which molecules changed classification (stable → unstable)
- Score deltas exceeding threshold
- Invariant violations

## 5.1 Benchmark Suite Format

Each line is a JSON object:

```
{"formula": "H2O", "seed": 42, "expected_class": "stable"}
{"formula": "CH4", "seed": 43, "expected_class": "stable"}
{"formula": "NaCl", "seed": 44, "expected_class": "ionic"}
...
```

## 5.2 Configuration Hashing

Every configuration gets a deterministic hash (lines 48–51):

```
def hash_config(self, config: Dict) -> str:
    """Hash␣config␣for␣change␣detection."""
    config_str = json.dumps(config, sort_keys=True)
    return hashlib.sha256(config_str.encode()).hexdigest()[:12]


config = {"bond_cutoff": 1.2, "angle_threshold": 0.1}
hash = "a3f4b7c8d1e2"
```

## 5.3 The Four Invariants

Lines 41–46:

```
INVARIANTS = {
    "energy_monotonic": "Energy␣should␣decrease␣during␣relaxation",
    "bonds_stable": "Bond␣count␣should␣stabilize␣after␣equilibration",
    "score_bounded": "Score␣should␣be␣in␣[0,␣1]",
    "classification_consistent": "Same␣input␣->␣same␣classification",
}
```

## 5.4 Comparison Algorithm

Lines 109–144:

```
def compare_runs(self, baseline: List[Result],
                 current: List[Result]) -> ChangeReport:
    """Compare two benchmark runs."""

    # Index by (formula, seed)
    baseline_map = {(r.formula, r.seed): r for r in baseline}
    current_map = {(r.formula, r.seed): r for r in current}

    # Find differences
    reclassified = []
    score_deltas = {}

    for key, curr in current_map.items():
        if key not in baseline_map:
            continue

        base = baseline_map[key]

        # Classification changed?
        if base.classification != curr.classification:
            reclassified.append({
                "formula": curr.formula,
                "seed": curr.seed,
                "old_class": base.classification,
                "new_class": curr.classification,
                "score_delta": curr.score - base.score,
            })

        # Score changed significantly?
        delta = abs(curr.score - base.score)
        if delta > 0.01:  # 1% threshold
            score_deltas[curr.formula] = curr.score - base.score

    # Check invariants
    invariant_breaks = self._check_invariants(current)

    return ChangeReport(
        config_hash_old="baseline",
        config_hash_new="current",
        config_diff="",
        total_runs=len(current),
        reclassified=len(reclassified),
        score_changed=len(score_deltas),
        invariant_breaks=invariant_breaks,
        reclassified_details=reclassified,
        score_deltas=score_deltas
    )
```

### 5.5 Invariant Checking

Lines 146–157:

```
def _check_invariants(self, results: List[Result]) -> List[str]:
```

```
    """Check if any invariants are broken."""

    breaks = []

    # score_bounded
    for r in results:
        if not (0.0 <= r.score <= 1.0):
            breaks.append(
                f"score_bounded: {r.formula} has score "
                f"{r.score:.3f} (out of [0,1])"
            )

    # TODO: Check other invariants (needs trajectory data)

    return breaks
```

### 5.6 Output Report

```
{
  "config_hash_old": "a3f4b7c8d1e2",
  "config_hash_new": "e9d2c1b7f4a3",
  "config_diff": "bond_cutoff: 1.2 -> 1.3",
  "total_runs": 100,
  "reclassified": 3,
  "score_changed": 12,
  "invariant_breaks": [
    "score_bounded: C6H6 has score 1.23 (out of [0,1])"
  ],
  "reclassified_details": [
    {
      "formula": "H2O",
      "seed": 42,
      "old_class": "stable",
      "new_class": "unstable",
      "score_delta": -0.23
    }
  ],
  "score_deltas": {
    "CH4": +0.015,
    "NH3": -0.022,
    ...
  }
}
```

### 5.7 Binary Verdict

The detector produces:

- **NO REGRESSIONS** — if reclassified = 0 and no invariant breaks
- **REVIEW REQUIRED** — otherwise

Parameter changes causing regressions are not accepted until:

1. The regressions are understood (root cause identified)
2. Either fixed or documented as acceptable trade-offs

# 6 The Self-Audit Loop

The three tools work together in a continuous improvement cycle:

| Tool | Input | Output |
|---|---|---|
| Failure Classifier | Error logs | Failure buckets + minimal repro |
| Gap Targeter | Coverage grid | Sparse/mismatch cells |
| Regression Detector | Config changes | Invariant breaks + verdict |

## 6.1 The Weekly Cycle

**Monday:** Run 10,000 simulations

- Failure classifier categorizes all failures
- UNKNOWN category analyzed: new patterns added

**Tuesday:** Gap analysis

- Gap targeter identifies sparse cells
- Schedule 1,000 runs to fill gaps

**Wednesday:** Execute gap-filling runs

- Update coverage metrics
- Check if 95% threshold reached

**Thursday:** Parameter tuning

- Propose scoring weight changes
- Run regression detector on benchmark suite

**Friday:** Review regressions

- If NO REGRESSIONS: accept changes
- If REVIEW REQUIRED: investigate, fix, or revert

# 7 Validation and Testing

## 7.1 Determinism Validation

**Test 1: Bit-identical runs**

```
# Run 10 times
for i in {1..10}; do
    ./build/vsepr-sim --formula H2O --seed 42 --output run_$i.xyz
    sha256sum run_$i.xyz >> hashes.txt
done

# Check uniqueness
uniq hashes.txt | wc -l  # Should be 1
```

**Pass criterion:** All 10 hashes identical.

**Test 2: Platform independence**

Run same command on Windows, Linux, macOS. Compare hashes.

**Current status:** Pass within platform, *not yet validated* across platforms.

## 7.2 Failure Classifier Validation

**Test:** Hand-label 100 failures, compare to classifier output.

**Metric:** Cohen's kappa $\kappa > 0.8$ (strong agreement).

**Current status:** Not yet performed (requires labeled dataset).

## 7.3 Gap Targeter Validation

**Test:** After 1000 iterations, check coverage:

$$\frac{n_{\text{cells with } n_{\text{runs}} \geq 10}}{600} > 0.95 \tag{5}$$

**Current status:** Framework implemented, not yet executed at scale.

## 7.4 Regression Detector Validation

**Test:** Inject known regression (break energy monotonicity), verify detector flags it.

**Current status:** Manual testing only.

# 8 Known Limitations and Future Work

## 8.1 Determinism: Cross-Platform

- Windows and Linux floating-point libraries differ (glibc vs. MSVCRT)
- `sin()`, `cos()`, `exp()` may vary in last bits
- Solution: Use fixed-precision math library (e.g., MPFR) or accept $\varepsilon = 10^{-12}$ tolerance

### 8.2 Failure Classifier: Pattern Coverage

- Current patterns based on manual inspection of ∼100 failures
- Needs validation on larger corpus (10,000+ failures)
- UNKNOWN fraction currently not monitored in production

### 8.3 Gap Targeter: Formula Generation

- Current formula selection is heuristic (lines 144–153)
- Does not adapt to chemistry (e.g., metals vs. organics)
- Better approach: Use composition library with verified structures

### 8.4 Regression Detector: Trajectory Analysis

- Energy monotonicity check requires trajectory data
- Current implementation only checks final results
- Extension: Store intermediate frames, check invariants at each step

## 9 Conclusion: The Self-Correcting Engine

This section has documented the self-audit infrastructure **as it exists in the codebase**:

- **Failure classifier:** 6 categories, regex pattern matching, minimal repro generation (250 lines Python)
- **Gap targeter:** 3D parameter grid (600 cells), mismatch detection, coverage-driven scheduling (200 lines)
- **Regression detector:** Config hashing, 4 invariants, binary verdict (200 lines)

The framework operates in a continuous loop:

1. Run simulations at scale (10,000+ runs)
2. Classify failures autonomously
3. Identify parameter space gaps
4. Schedule targeted runs to fill gaps
5. Detect regressions when parameters change
6. Accept changes only if no invariants break

This is **not a theoretical proposal**. The tools exist, have clear interfaces, and integrate into the formation pipeline. What remains is *execution at scale*—running the full loop for weeks and validating the 95% coverage criterion.

The principle: **trust but verify**. The formation engine does not assume its results are correct. It continuously audits itself, flags anomalies, and demands human review when invariants break. This is the difference between a research prototype and a production scientific instrument.

**Transition to §12:** With the self-audit infrastructure established (failure classification, gap targeting, regression detection), Section 12 documents the *validation doctrine*: the hierarchical test suite that certifies the formation engine for production use.